



Connect AI readings and WhatsApp verification

A step-by-step guide for the client account owner

Follow these pages to prepare the OpenAI and MSG91 accounts for Star Talks, then give the required details to your developer. You retain ownership of both accounts and their billing.

Service	What it provides	Your final deliverable
OpenAI	AI readings using gpt-4o-mini	A funded API project and its application API key
MSG91 + Meta	WhatsApp codes for phone verification	Authkey, active sender, approved authentication template and its API details

Have these ready

- A business email address and access to its inbox.
- A payment method for provider charges, with a budget you approve.
- Administrator access to your Meta business portfolio, business information and verification documents if requested.
- A business phone number you control that can receive a verification call or SMS, plus a separate consumer WhatsApp number for delivery testing.

Check WhatsApp eligibility early. MSG91 says Meta restricts authentication templates to eligible businesses. A newly created account may need further verification or eligibility review. Ask MSG91 to confirm your account can send authentication messages before committing to a paid plan.

Who does what

You create and fund the provider accounts, complete business verification, and approve the WhatsApp template. Your developer configures the application, generates the internal OTP secret, and tests the complete app flow.

Use a desktop browser. Dashboard labels may vary by account. Links in this PDF are clickable. Account approvals and message delivery cannot be guaranteed by completing a form.

Reference: [MSG91 authentication eligibility](#) · [Selected OpenAI model](#)

Create and fund the API project

1. Sign in and select the business account

Open platform.openai.com. Sign up or sign in with the client-owned account and finish any requested verification. Select the organization that should own Star Talks usage and billing.

2. Create a dedicated project

Open the project selector and choose **Create project**. Name it **Star Talks**. If project creation is unavailable, ask the organization owner to create it or grant the required access.

3. Activate API billing

1. Open [API billing overview](#) for the correct organization.
2. Select **Add payment details** and enter the client's billing information.
3. Purchase the initial API credit amount shown by the dashboard. OpenAI currently documents a US\$5 minimum for prepaid setup; choose an amount you approve.
4. Review **Use auto-reload** before confirming. Disable it if you want to approve each purchase. If enabled, choose the recharge threshold, amount and monthly reload limit.
5. Wait for the dashboard to show available credit or active billing.

API billing is separate from ChatGPT. A ChatGPT subscription does not fund the API calls made by Star Talks. Record the approved budget and who will monitor usage.

4. Set model access and spending controls

In the Star Talks project's **Limits / Model Usage** settings, allow gpt-4o-mini. The application also uses the moderation endpoint with omni-moderation-latest. Ask your developer to check any missing permissions.

Set a monthly spend limit and alerts. Where the dashboard offers enforcement, choose whether the limit should stop requests or only notify you. An alert alone does not stop usage; a prepaid balance is not an instantaneous cutoff.

Official help: [Projects and model permissions](#) · [Prepaid billing](#) · [Separate API billing](#) · [Spend limits](#)

Create the application key

5. Create a project service account

1. Select the **Star Talks** project.
2. Go to project settings → **Members** → **+ Service account**.
3. Name it star-talks-backend and create it.
4. Copy the generated secret into your password manager immediately. The full value is displayed only at creation.

If the service-account option is unavailable, ask the organization owner to create it. A project API key can also be created under **API Keys** → **Create new secret key**; keep the key dedicated to Star Talks.

6. Check the key's permissions

In the project's API Keys settings, use **Restricted** permissions where available. The backend needs permission to create responses (POST /v1/responses) and submit moderation requests (POST /v1/moderations). A read-only key cannot run these calls. Give your developer the permission summary if the dashboard labels differ.

The value to hand over is the complete secret API key. The project name, project ID, key name and masked key shown later in the dashboard cannot replace it. If the secret was lost, create a replacement.

7. Hand over these OpenAI details

Item	What to provide
OPENAI_API_KEY	The application key through the agreed secure channel
Project	Star Talks project name and ID, if shown
Billing	Confirmation that billing is active, plus the approved testing budget
Access	Confirmation of model and endpoint permissions

Your developer will make a small text request, an image request and a moderation request. A successfully created key alone does not prove that billing and model access are ready.

Official help: [Project service accounts and API keys](#) · [Finding and saving a key](#) · [API permissions](#)

Connect the WhatsApp sender

8. Create the MSG91 account

Open [MSG91](#), sign up or sign in using the client's business details, verify the registered contact information, and complete any requested KYC steps.

9. Confirm authentication-template eligibility

Contact MSG91 support from the dashboard with the message below. Complete any Meta business verification or eligibility steps they identify before purchasing a plan for this use case.

Message to MSG91 support

"We are launching Star Talks, a mobile application. We need to send six-digit phone-verification codes using your direct WhatsApp authentication-template API. Please confirm our business/sender is eligible, the required verification steps, supported destination countries, all subscription and message charges, and any minimum spend."

Prepare Meta: open business.facebook.com, sign in with your Facebook account, and use Business Settings → Create Business Portfolio if needed. Enter the legal business name, contact and email, then confirm the email. In Business Settings → Security Center, follow the business-verification flow offered. Supply the requested legal-name/address documents and complete the contact verification. If the option is unavailable, ask MSG91 which verification path applies.

10. Add the business number

1. In MSG91, open **WhatsApp** → **Numbers** → **Add Number**.
2. Choose the onboarding option for your number. For a fresh dedicated number, use the new-number flow. For an existing WhatsApp Business App number, ask MSG91 about its supported coexistence or migration flow before changing the current account.
3. Sign in to Meta with the account that administers the business portfolio. Select the correct portfolio and create or select its WhatsApp Business Account.
4. Enter the business/display name and category, add the number with its country code, and complete the SMS or voice verification offered.
5. Review the connection permissions and finish onboarding. Wait until MSG91 shows the number as active and any required display-name/business review has completed.

11. Fund WhatsApp delivery

Confirm the quoted WhatsApp plan and destination-country rates, then add the required wallet balance. Choose recharge settings you approve. Keep a record of recurring fees, per-message charges and taxes; the earlier cost overview is a budget estimate.

Official help: [Account and authentication eligibility](#) · [Number onboarding](#) · [Meta portfolio](#) · [Business verification](#) · [Onboarding options and wallet](#) · [Current pricing](#)

Approve and test the OTP template

12. Create the authentication template

1. Open **WhatsApp** → **Templates** → **Create Template**.
2. Choose the active sender. Suggested name: `star_talks_phone_verification`.
3. Select category **Authentication**. Start with English and record the exact language code shown in the generated API example.
4. Use the provider's authentication format with a code variable. Choose a copy-code delivery option if offered; Star Talks currently lets the user enter the code manually. Leave native autofill/zero-tap integration for the developer.
5. If an expiry display is available, set it to **5 minutes**, matching the application's code lifetime. Review the preview and submit for Meta approval.
6. Wait for **Approved** status. If rejected, use the displayed reason or MSG91 support to correct it and resubmit.

13. Save the template's generated API details

On the approved template, open the **Code / API / cURL** option. Give the developer a copy with any real authkey and recipient number removed. This shows the exact `integrated_number`, template name, language code, namespace and component mapping.

Copy values exactly. The sender is the integrated WhatsApp phone number, not the recipient number or a numeric template ID. Use the namespace from the generated example. If it is missing, ask MSG91 support; do not guess a value.

14. Send a real delivery test

Open **Send WhatsApp** → **Manually**, choose the sender and approved template, enter a consenting consumer WhatsApp number with country code, fill the example code variable, and send. Check both the recipient's inbox and MSG91's delivery report. Record the test time and delivery status for the developer.

This checks provider delivery. The developer will separately test the real app-generated code, expiry, resend and account verification. Use a consumer WhatsApp recipient; MSG91 documents restrictions on authentication delivery to other WhatsApp Business Accounts.

Official help: [Template creation and Code option](#) · [Required API fields and recipient restrictions](#) · [Manual message testing](#)

Create the Authkey and send the details

15. Create the MSG91 Authkey

Open **Authkey** in MSG91's top menu or account menu. Complete the registered-phone OTP check, choose **Create New**, and name the key StarTalksBackend. Select the integration use case and a rule allowing WhatsApp template sending. Create and securely save the key.

MSG91 enables IP security by default. Tell the developer whether it is enabled and share the rule settings. The developer must arrange a compatible server egress policy with MSG91; whitelisting only your laptop's IP will not authorize the app's backend.

16. Complete this handover checklist

Required item	Where to get it / status
OPENAI_API_KEY	OpenAI service account or project key
MSG91_AUTH_KEY	MSG91 Authkey created above
MSG91_WHATSAPP_SENDER	Exact integrated_number from cURL
MSG91_WHATSAPP_TEMPLATE	Approved template name
MSG91_WHATSAPP_NAMESPACE	Template namespace from cURL
MSG91_WHATSAPP_LANGUAGE	Exact template language code
Supporting details	Redacted cURL; approved template screenshot; sender status; delivery result; IP rule settings; funded billing and agreed budgets

Send secrets through an agreed password-manager share or secure secret link. Keep them out of this PDF, Git, public screenshots and ordinary email. Keep account passwords, payment details and login OTPs private. If the developer has project access, you may enter the keys directly into the project's server secret settings instead.

The developer generates PHONE_OTP_PEPPER; you do not obtain it from a provider. A widget token, Meta login password or OpenAI Admin API key is not needed for this integration.

If you get stuck

Missing key/billing controls: check the selected account and owner permissions. Missing Authentication category or failed delivery: ask MSG91 to review eligibility, template, sender and wallet status. Send the developer the error text with secret values removed.

Official help: [Authkey creation and IP rules](#) · [Server credential handling](#)